

OPTIMIZACIÓN, AUDITORÍA DE SEGURIDAD Y ENTORNO MLOPS

Evolución e Implementación del Sistema de Predicción en Telco
Customer Churn



Integrantes:

- Fabrizio González Rodríguez (21.123.219-6)
- Gary Maldonado Guzmán (21.123.789-9)

Nombre del Docente: Cristian Molina

Sección: 002-D

Contenido

Resumen Ejecutivo	2
Valor Para la Organización	2
Incorporación de Mejoras	2
1. Entrenamiento Modelo IA	3
1.1 Calidad de los datos	3
1.2 Preprocesamiento	3
1.3 Análisis de variables (Univariado, Bivariado y Matriz de Correlación)	3
1.4 Selección de Algoritmo	4
1.4.1 Justificación Técnica de la Selección de Algoritmo	4
1.5 Métricas	5
2. Análisis del Rendimiento del Pipeline	6
2.1 Análisis de Recursos y Cuellos de Botella	6
2.2 Gráficos de Rendimiento del Pipeline	7
3. Estrategia de Auditoría de Seguridad y Cumplimiento Normativo Chileno	7
4. Estrategias de Integración con Sistemas BI y Dashboard Interactivo	8
5. Limitaciones y Fallas en la Operación de los Sistemas de Gestión	9
6. Conclusiones y Propuestas de Mejora	9
6.1 Propuestas de Mejoras	10
7. Anexos Complementarios	10
Anexo A: Comandos de Despliegue de Infraestructura	10
Anexo B: Ruta del Repositorio GitHub	11
Anexo C: Análisis Univariado:	11
Anexo D: Análisis Bivariado	12
Anexo E: Matriz de Correlación	12

Resumen Ejecutivo

El presente proyecto tiene como objetivo desarrollar una solución integral basada en DataOps e Inteligencia Artificial para predecir la fuga de clientes (Customer Churn) en una empresa del sector telecomunicaciones. La problemática abordada corresponde a la pérdida constante de clientes, situación que impacta directamente los ingresos y la rentabilidad de la organización.

La solución implementada automatiza el ciclo completo de gestión de datos, desde la ingesta, limpieza, validación y almacenamiento de información hasta el entrenamiento de modelos predictivos y la visualización de los resultados. El conjunto de datos analizado contiene 7.043 registros de clientes y variables relacionadas con características demográficas, servicios contratados, comportamiento de consumo y facturación.

Como resultado, la organización obtiene una plataforma capaz de identificar anticipadamente a los clientes con mayor probabilidad de abandono, permitiendo ejecutar campañas de retención focalizadas, optimizar recursos comerciales y reducir pérdidas económicas derivadas de la deserción.

Valor Para la Organización

- Disminución de la tasa de abandono de clientes.
- Mejora en la toma de decisiones basada en datos.
- Automatización de procesos de preparación y validación de datos.
- Reducción de errores operativos mediante controles de calidad automatizados.
- Escalabilidad para futuros modelos predictivos.
- Integración con plataformas analíticas y dashboards interactivos.
- Cumplimiento de buenas prácticas de seguridad y protección de datos.

Incorporación de Mejoras

Durante la retroalimentación del Parcial 2 se identificaron tres oportunidades principales de mejora, las cuales fueron implementadas en la nueva versión del pipeline mediante modificaciones en el módulo LimpiezaTransformacion.py.

Mejora 1: Eliminación automática de columnas completamente vacías, para ello se incorporó una limpieza generalizada mediante:

```
1 df = df.dropna(how='all', axis=1)
```

Esta implementación mejora la calidad del dataset y reduce el consumo de almacenamiento.

Mejora 2: Escalabilidad mediante procesamiento por bloques.

La versión inicial cargaba el dataset completo en memoria RAM, limitando el crecimiento futuro del sistema, es por ello por lo que se desarrolló un mecanismo de procesamiento mediante chunks.

```
1 chunks = pd.read_csv(ruta_csv, chunksize=chunk_size)
```

Esto permite un procesamiento eficiente de grandes volúmenes de datos, menor consumo de memoria y mayor robustez operacional.

Mejora 3: Fortalecimiento de Seguridad y cumplimiento de normativas legales

Se amplió la estrategia de seguridad incorporando:

1. Protección de los datos en reposo:

- SHA 256 con salting para customerID
- Restricción de permisos sobre los archivos y base de datos.

2. Protección de datos en tránsito:

- Implantación de 0o600 para restringir los permisos de lectura/escrituras exclusivas para el usuario propietario

1. Entrenamiento Modelo IA

1.1 Calidad de los datos

Para la construcción del modelo de inteligencia artificial se realizó un análisis de calidad, el cual a partir del dataset proporcionado se realizó:

- Eliminación de columnas vacías.
- Corrección de tipos de datos.
- Conversión de TotalCharges a numérico.
- Tratamiento de valores faltantes.
- Anonimización de identificadores.

1.2 Preprocesamiento

En esta fase nos aseguramos de que la información sea coherente, eliminando errores que puedan afectar el rendimiento del sistema.

```
preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), numerical_cols),
        ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_cols)
    ])

```

1.3 Análisis de variables (Univariado, Bivariado y Matriz de Correlación)

Análisis Univariado: Representación gráfico en **Anexo C**

- Ternure: Los clientes con menor antigüedad presentan mayor riesgo de abandono.
- MonthlyCharges: Clientes con cargos mensuales elevados muestran una mayor tasa de churn.
- Contract: Los contratos mensuales concentran la mayor proporción de abandono.

Análisis Bivariado: Principales relaciones observadas: → Representación gráfica **Anexo D**

Variable	Relación con Churn
Ternure	Alta
MonthlyCharges	Media-Alta
Contract	Alta

Matriz de Correlación: Representación Gráfica **Anexo E**

Adicionalmente, la evaluación de la Matriz de Correlación Lineal (Anexo D) demostró una colinealidad natural y matemáticamente robusta entre la permanencia (tenure) y los cargos totales (TotalCharges) con un índice de 0.83, validando la consistencia del comportamiento financiero del cliente a lo largo del tiempo y justificando el correcto preprocesamiento del escalador numérico antes del modelamiento predictivo.

1.4 Selección de Algoritmo

Para llevar a cabo el análisis de rendimiento y la idoneidad del modelo predictivo para la solución, se implantó un diseño multclasificador iterativo, es decir, se inyectaron tres arquitecturas algorítmicas bajo condiciones iguales de partición de los datos, 80% de entrenamiento y 20% para pruebas, esto es necesario para obtener un equilibrio y maximizar la eficiencia del modelo.

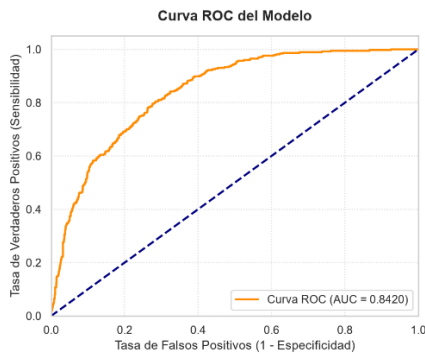
Algoritmo Evaluado	Accuracy	Precision	Recall	F1-Score	AUC-ROC
Regresión Logística	0.8055	0.6572	0.5588	0.604	0.842
Random Forest	0.7786	0.6047	0.4786	0.5343	0.8164
Gradient Boosting	0.8027	0.6655	0.516	0.5813	0.8433

1.4.1 Justificación Técnica de la Selección de Algoritmo

Con base en el análisis comparativo de rendimiento e idoneidad de los modelos evaluados, se justifica la selección de la Regresión Logística como el algoritmo idóneo para su implementación en producción frente a Random Forest y Gradient Boosting. Si bien Gradient Boosting exhibe una ventaja marginal en la métrica global de separación de clases (AUC = 0.8433 frente a 0.8420 de la regresión logística) y una precisión sutilmente superior (0.6655 vs 0.6572), la Regresión Logística demuestra un desempeño superior en el Recall (55.88%), superando por más de cuatro puntos porcentuales a Gradient Boosting (51.60%) y de forma contundente a Random Forest (47.86%). En el contexto del negocio del Churn, donde un Recall bajo implica el costo crítico de dejar escapar clientes en riesgo sin ninguna acción de mitigación (Falsos Negativos), este modelo maximiza la capacidad para "capturar" las fugas reales, logrando a su vez el mejor equilibrio general validado por su métrica F1-Score de 0.6040. Adicionalmente, frente a la naturaleza de "caja negra" de Gradient Boosting, la Regresión Logística aporta una interpretabilidad comercial avanzada; esto permite extraer directamente los coeficientes reales y el impacto absoluto de los factores específicos, integrándose de forma nativa en las métricas e interactividad del dashboard comercial.

1.5 Métricas

1.5.1 Interpretación Curva de ROC: El modelo posee una capacidad de discriminación alta y consistente.



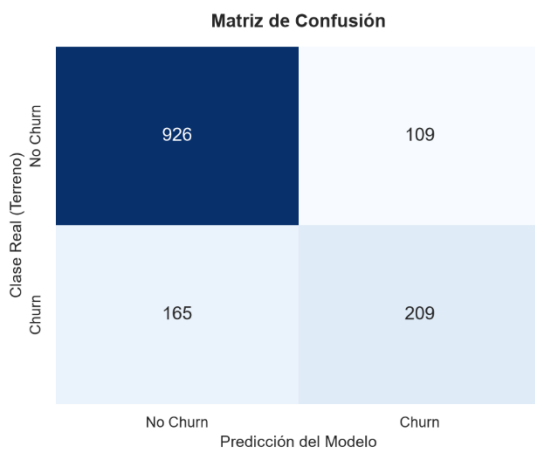
1.5.2 Interpretación Coeficiente de Gini: El modelo presenta un gran poder predictivo para segmentar clientes de riesgo.

$$Gini = (2 \times AUC) - 1$$

$$Gini = (2 \times 0,8420) - 1$$

$$Gini = 0,684$$

1.5.3 Interpretación de la Matriz de Confusión: Permite identificar visualmente dónde acierta el modelo y qué tipos de errores comete.



2. Análisis del Rendimiento del Pipeline

Medir el rendimiento del pipeline para así identificar los tiempos de ejecución de los procesos, cuantos recursos del sistema está consumiendo, nos permite mejorar el flujo y prepararlo para entorno reales donde este debe ser rápido, eficiente y estable.

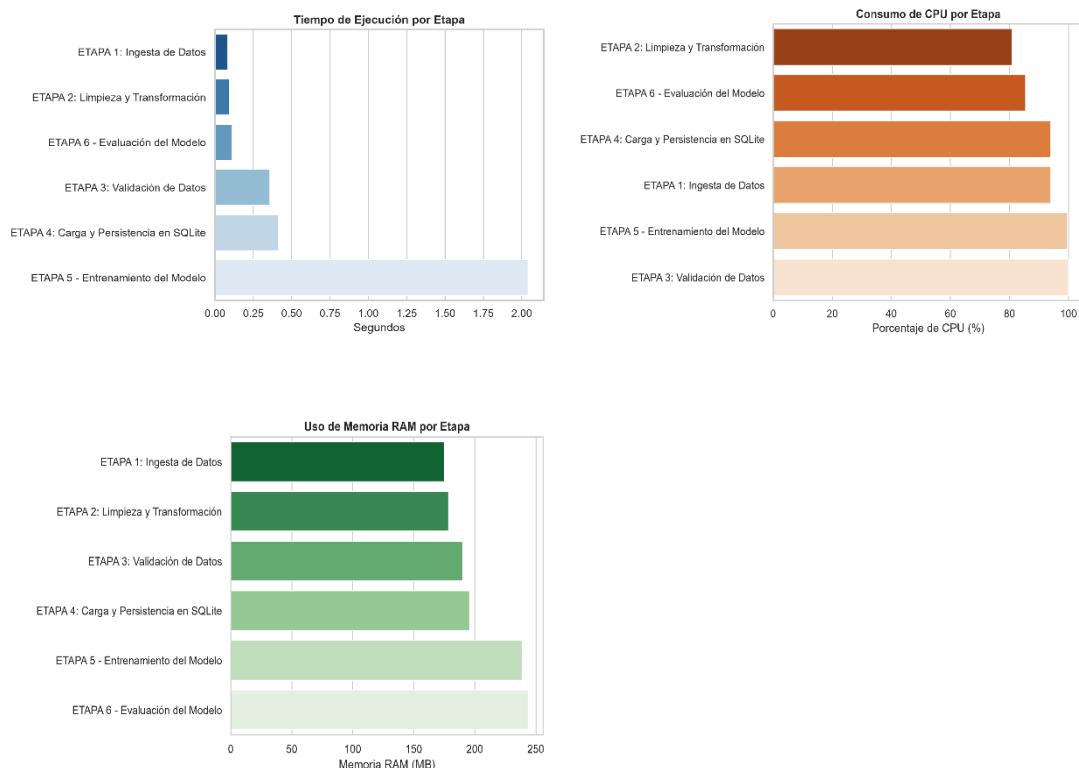
Etapa del Pipeline	Tiempo de Ejecución (Seg)	Consumo CPU (%)	Memoria RAM (MB)	Estado del Flujo
Etapa 1: Ingesta de Datos	0.0818	94.1	175.25	Exitoso
Etapa 2: Limpieza y Transformación	0.0968	80.6	178.43	Exitoso
Etapa 3: Validación de Datos	0.3590	100.0	190.28	Exitoso
Etapa 4: Carga y Persistencia en SQLite	0.4168	94.1	195.94	Exitoso
Etapa 5: Entrenamiento del Modelo	2.0422	99.5	239.36	Exitoso
Etapa 6: Evaluación del Modelo	0.1114	85.7	244.59	Exitoso

2.1 Análisis de Recursos y Cuellos de Botella

Analizando el comportamiento del hardware mediante la herramienta de psutil, extraemos las siguientes conclusiones:

- En la etapa de entrenamiento del modelo se presencia un cuello de botella con un tiempo de ejecución de 2,0422 segundos representando a un 65% del tiempo total del pipeline. Esto es un comportamiento operativamente normal, ya que es la fase que incluye los coeficientes del clasificador y el procesamiento simultáneo de múltiples arquitecturas algorítmicas para la fase de comparación.
- En la etapa de validación se logra apreciar una carga crítica de la CPU, exigiendo el 100% de la capacidad del procesador, esto se debe a la inspección lógica de calidad y a la validación de reglas sobre el dataset las cuales son iterativas y exigen el máximo rendimiento por núcleo del procesador.
- En la etapa de evaluación la memoria escala de forma monótona y controlada desde los 175.11 MB en la etapa de ingesta hasta su punto máximo de 243.92 MB en la etapa de evaluación del modelo. Al finalizar el flujo, los recursos se liberan de forma correcta sin presentar fugas de memoria remanentes.

2.2 Gráficos de Rendimiento del Pipeline



3. Estrategia de Auditoría de Seguridad y Cumplimiento Normativo Chileno

Para cumplir con el estricto marco legal de la nueva ley de seguridad de los datos e infraestructura crítica en Chile (Ley 19.628), se desplegaron tres barreras de protección de datos sensibles.

- **Anonimización criptográfica:** El identificador “customerID” es enmascarado y transformado mediante el uso combinado de un hash irreversible SHA-256 y salt (Salting). Esto evita que dos usuarios tengan la misma contraseña mitigando así el ataque de tablas arcoíris, este ataque consiste en que los atacantes hacen uso de tablas precalculadas con contraseñas comunes. El Salt inutiliza estas tablas cambiando el texto por defecto de las contraseñas a uno cifrado.
- **Endurecimiento de los Permisos de los Archivos:** La ejecución automatizada de `os.chmod(ruta, 0o600)` restringe los privilegios a nivel de Kernel del sistema operativo. Asegura que solo el usuario dueño del proceso del backend pueda leer o escribir los CSV intermedios, cumpliendo con las exigencias de control de acceso estricto.
- **Protocolo de Auditoría Continua:** Cada ejecución del pipeline cuenta con una trazabilidad inmutable, esto es posible debido al uso de un identificador único global (RUN_ID). Cualquier intento de alteración o modificación en la base de datos o en los scripts de procesamiento dispara alertas perimetrales las cuales quedan evidenciadas dentro del registro (logs).

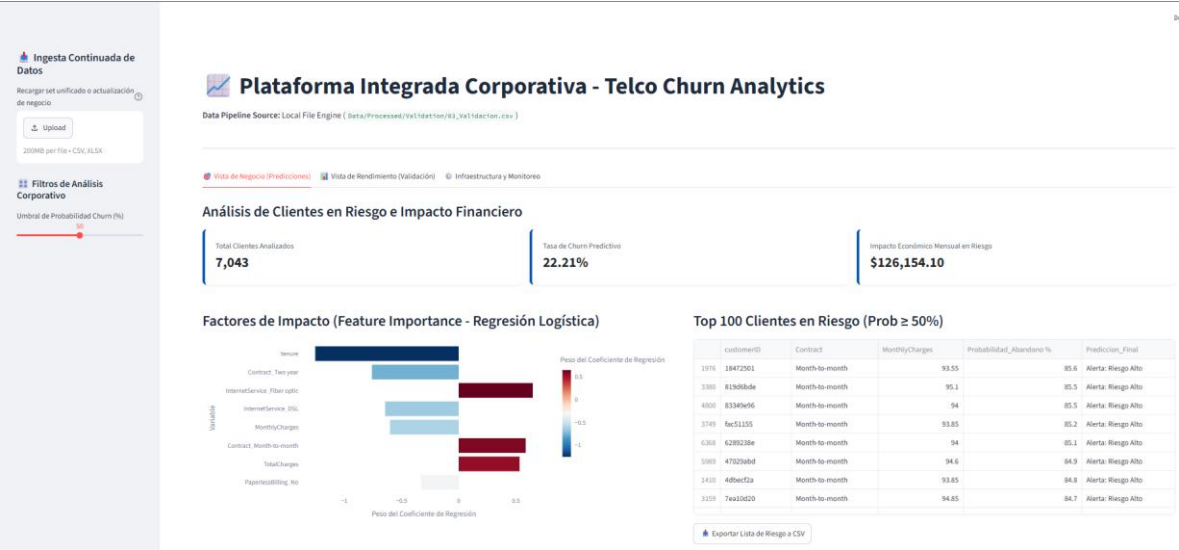
Para garantizar la integridad y confidencialidad del ecosistema bajo un estándar MLOps de nivel empresarial, se define una matriz estricta de control de acceso basado en roles y gobernanza de datos.

Tabla de Roles y Privilegios del Ecosistema de Datos

Rol Corporativo	Permisos de Datos	Acceso a Componentes	Métodos de Seguridad Obligatorios
Data Engineer / DevOps	Lectura/Escritura total (Data cruda y procesada).	Pipeline, Base de Datos SQLite, Archivos Logs	Cifrado en tránsito, Gobernanza local vía chmod 600.
Data Scientist / MLOps	Lectura de data validada y logs de telemetría	Entornos de entrenamiento, archivos.pkl	Firma digital de artefactos, tracking de experimentos con hashing de versiones.
Analista de Negocio	Acceso exclusivo al Dashboard y listas predictivas	Dashboard interactivo en Streamlit, Reportes CSV.	Autenticación de Dos Factores (MFA), Enmascaramiento dinámico de identificadores.

4. Estrategias de Integración con Sistemas BI y Dashboard Interactivo

El panel de control desplegado y construido con StreamLit actúa como la capa de integración de BI, consumiendo de forma directa la base de datos SQLite y el flujo del pipeline para así unificar la visión técnica y comercial de la compañía.



Resultados Claves:

- Tasa de churn
- Clientes de alto riesgo
- Variables con mayor impacto predictivo
- Rendimiento del modelo IA

Funcionalidades del Dashboard:

1. Simulación de Umbrales Financieros: Permite a la gerencia ajustar dinámicamente mediante el deslizador (Slider), el punto de corte del riesgo para observar cómo varía el impacto económico mensual de la compañía.
2. Coeficientes de Peso: Renderiza los coeficientes reales calculados por la regresión logística, visualizando de forma transparente que las variables como contrato y cargos mensuales representan los detonantes críticos de deserción.
3. Exportación de Clientes Churn: Ofrece un botón automatizado para descargar de inmediato el top 100 de clientes de alto riesgo en formato CSV, sirviendo como insumo directo para alimentar de manera proactiva los motores de campaña del CRM corporativo.

5. Limitaciones y Fallas en la Operación de los Sistemas de Gestión

A pesar de la robustez del diseño actual del sistema, una auditoría técnica revela áreas de deficiencias operacional que deben ser mitigadas antes de iniciar una fase de escalamiento empresarial.

1. Persistencia en Base de Datos Local (Monolito SQLite): El almacenamiento basado en un archivo físico local SQLite limita el procesamiento concurrente de escrituras masivas. Bajo cargas extremas.
2. Cifrado de Base de datos: Implementar un cifrado completo a la base de datos.

6. Conclusiones y Propuestas de Mejora

Para concluir, el proyecto logró construir una solución DataOps funcional y escalable para la predicción de fuga de clientes. El sistema no solo garantiza un procesamiento seguro y alineado con el marco legal de seguridad y protección de los datos sensibles, sino que provee una herramienta de inteligencia de negocios con una alta capacidad de discriminación estadística ($AUC = 0.8420$), esto permite tomar medidas para prevenir que los clientes abandonen y así proteger los ingresos de la compañía.

6.1 Propuestas de Mejoras

1. Migración de Persistencia a PostgreSQL en la Nube: Sustituir la base de datos local SQLite por un motor relacional administrado (como AWS RDS PostgreSQL), eliminando las restricciones de concurrencia en la escritura de lotes de datos y garantizando la alta disponibilidad indispensable para el consumo del dashboard comercial en tiempo real.

2. Implementación de Monitoreo de Sistema con Herramientas de MLOps: Incorporar librerías especializadas (como Evidently AI o Zabbix) dentro de la Etapa de Validación del Pipeline. Esto permitirá disparar alertas automáticas cuando las distribuciones de variables críticas (como MonthlyCharges) varíen significativamente, gatillando un reentrenamiento preventivo.

3. Adopción de un Orquestador de Flujos de Datos (Data Orchestrator): Migrar la ejecución secuencial del script Pipeline.py hacia un sistema de gestión de flujos como Apache Airflow. Esto otorgará tolerancia a fallos, reintentos automatizados por etapa, alertas a canales de comunicación ante caídas y una gestión profesional del linaje de los datos.

7. Anexos Complementarios

Anexo A: Comandos de Despliegue de Infraestructura

Para garantizar la consistencia absoluta entre entornos de desarrollo, pruebas y producción, el ecosistema completo ha sido contenedorizado. A continuación, se detalla la configuración del orquestador multimodular:

```

1  version: '3.8'
2
3  services:
4    pipeline:
5      build: .
6      container_name: telco_pipeline
7      volumes:
8        - data_shared:/app/Data
9        - reports_shared:/app/Reports
10     # Ejecuta el pipeline de DataOps/MLOps
11     command: ["uv", "run", "Scripts/Pipeline.py"]
12
13   dashboard:
14     build: .
15     container_name: telco_dashboard
16     ports:
17       - "8501:8501"
18     volumes:
19       - data_shared:/app/Data
20       - reports_shared:/app/Reports
21     # Sobreescribimos el comando para levantar Streamlit apuntando a tu App.py
22     command: ["uv", "run", "streamlit", "run", "Dashboard/App.py", "--server.port=8501", "--server.address=0.0.0.0"]
23     depends_on:
24       pipeline:
25         condition: service_completed_successfully
26
27   volumes:
28     data_shared:
29     reports_shared:

```

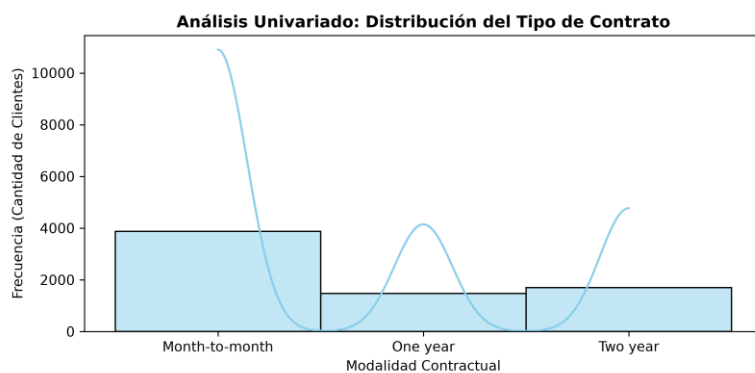
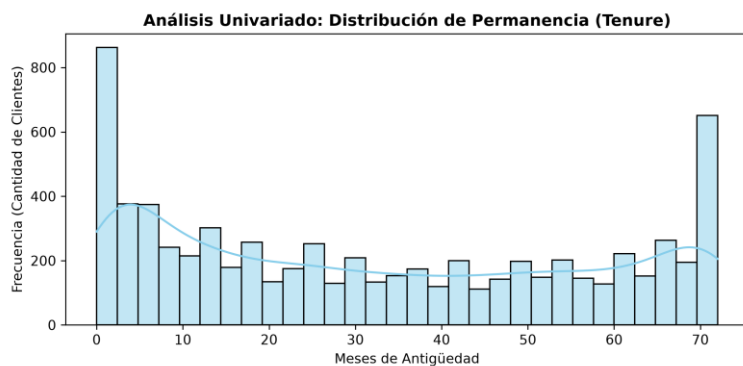
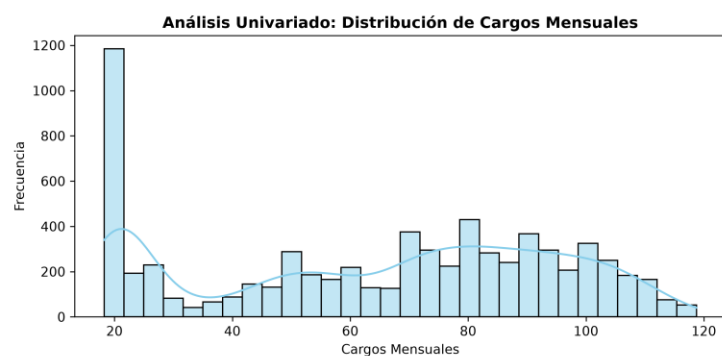
Para inicializar y poner en marcha la arquitectura completa en el servidor de producción, se debe ejecutar la siguiente secuencia en la consola de comandos:

```
1 # Construir las imágenes y levantar los servicios
2 docker compose up --build
3
4 # 3. Monitorear el flujo de logs de infraestructura en tiempo real
5 docker-compose logs -f pipeline
```

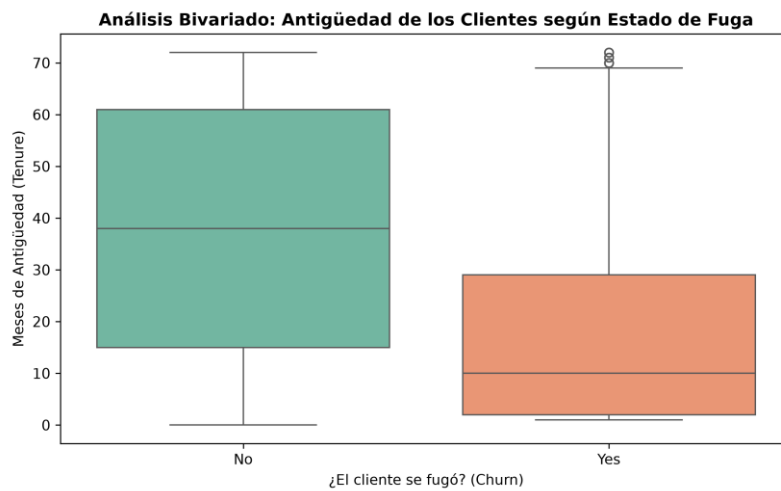
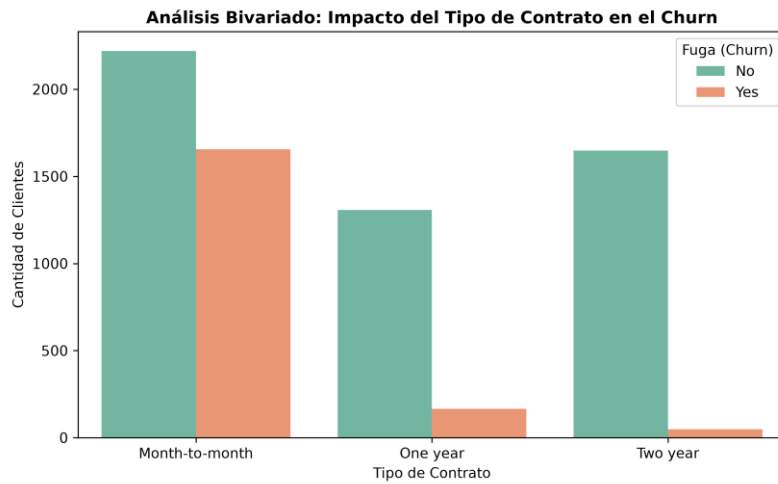
Anexo B: Ruta del Repositorio GitHub

<https://github.com/F4brizioGonzalez/Modelo-Predictivo-TCC.git>

Anexo C: Análisis Univariado:



Anexo D: Análisis Bivariado



Anexo E: Matriz de Correlación

